

Relevant Links

By: Magnus Vaslag Løvøy Github: <https://github.com/magloev/magloevIND320> Streamlit app: <https://magloevind320.streamlit.app/>

Renaming and Sorting Dataset

```
In [7]: import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('../data/reservoirs.csv')

#Renaming the columns to reasonable English equivalents
df = df.rename(columns= {
    "dato_Id" : "date",
    "omrType" : "area_type",
    "omrnrr" : "area_id",
    "iso_aar" : "iso_year",
    "iso_uke" : "iso_week",
    "fyllingsgrad" : "fill_level",
    "kapasitet_TWh" : "capacity_TWh",
    "fylling_TWh" : "stored_energy_TWh",
    "neste_Publiseringdato" : "next_publication_date",
    "fyllingsgrad_forrige_uke" : "fill_level_prev_week",
    "endring_fyllingsgrad" : "fill_level_change" })

#Sort the data by date
df["date"] = pd.to_datetime(df["date"])
df = df.sort_values(by="date")

df.tail(20)
```

Out[7]:

	date	area_type	area_id	iso_year	iso_week	fill_level	capacity_TWh	stored_energy_TWh	next_publication_date	fill_level_prev_w
8252	2026-08-23	EL	5	2026	34	0.641755	17.390587	11.160489	2026-09-02T13:00:00	0.6534
14849	2026-08-23	VASS	1	2026	34	0.570732	36.044464	20.571724	2026-09-02T13:00:00	0.5836
14874	2026-08-30	VASS	3	2026	35	0.827718	28.155403	23.304743	2026-09-09T13:00:00	0.8298
8235	2026-08-30	EL	2	2026	35	0.459006	34.043590	15.626217	2026-09-09T13:00:00	0.4662
9916	2026-08-30	NO	0	2026	35	0.634381	87.437580	55.468720	2026-09-09T13:00:00	0.6395
8257	2026-08-30	EL	5	2026	35	0.635164	17.390587	11.045877	2026-09-09T13:00:00	0.6417
8255	2026-08-30	EL	4	2026	35	0.906874	21.079208	19.116190	2026-09-09T13:00:00	0.9075
8259	2026-08-30	EL	3	2026	35	0.627002	8.920932	5.593442	2026-09-09T13:00:00	0.6372
14858	2026-08-30	VASS	2	2026	35	0.514370	23.237715	11.952795	2026-09-09T13:00:00	0.5252
14856	2026-08-30	VASS	1	2026	35	0.567064	36.044464	20.439535	2026-09-09T13:00:00	0.5707
8258	2026-08-30	EL	1	2026	35	0.716472	6.003264	4.301168	2026-09-09T13:00:00	0.7126
8260	2026-09-06	EL	1	2026	36	0.725045	6.003264	4.352634	2026-09-16T13:00:00	0.7164
8261	2026-09-06	EL	5	2026	36	0.638898	17.390587	11.110812	2026-09-16T13:00:00	0.6357
9917	2026-09-06	NO	0	2026	36	0.636054	87.437580	55.615055	2026-09-16T13:00:00	0.6344

	date	area_type	area_id	iso_year	iso_week	fill_level	capacity_TWh	stored_energy_TWh	next_publication_date	fill_level_prev_wk
8262	2026-09-06	EL	3	2026	36	0.632579	8.920932	5.643190	2026-09-16T13:00:00	0.6270
8263	2026-09-06	EL	4	2026	36	0.902762	21.079208	19.029512	2026-09-16T13:00:00	0.9069
14875	2026-09-06	VASS	1	2026	36	0.570397	36.044464	20.559645	2026-09-16T13:00:00	0.5670
14873	2026-09-06	VASS	3	2026	36	0.826252	28.155403	23.263466	2026-09-16T13:00:00	0.8276
8264	2026-09-06	EL	2	2026	36	0.460863	34.043590	15.689445	2026-09-16T13:00:00	0.4589
14876	2026-09-06	VASS	2	2026	36	0.516978	23.237715	12.013381	2026-09-16T13:00:00	0.5144

Plotting

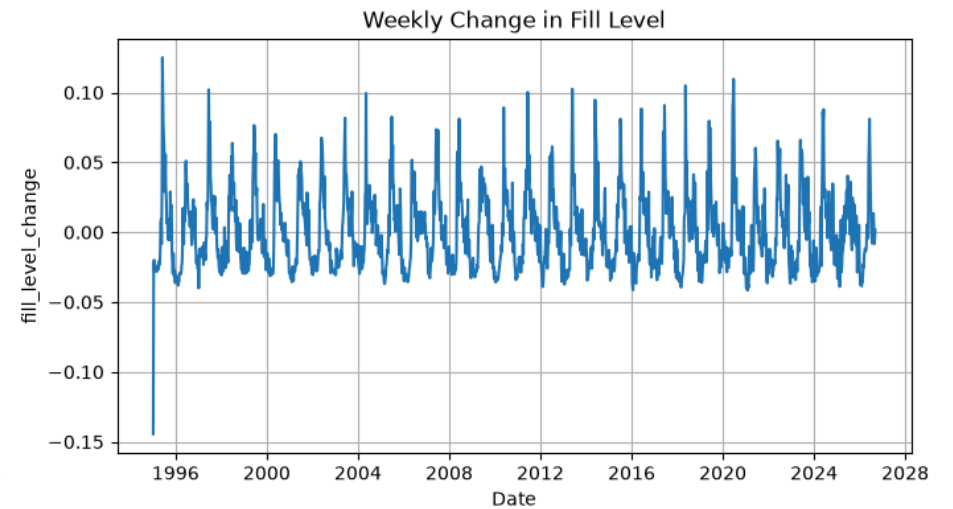
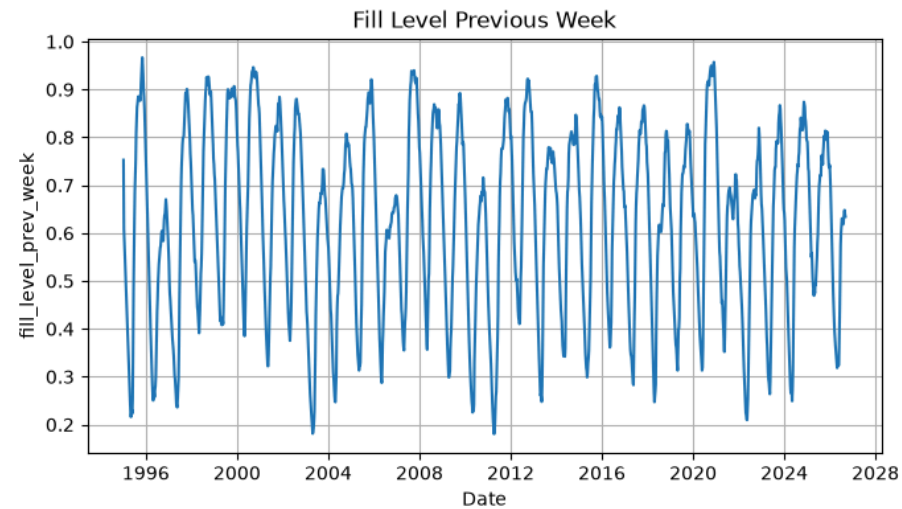
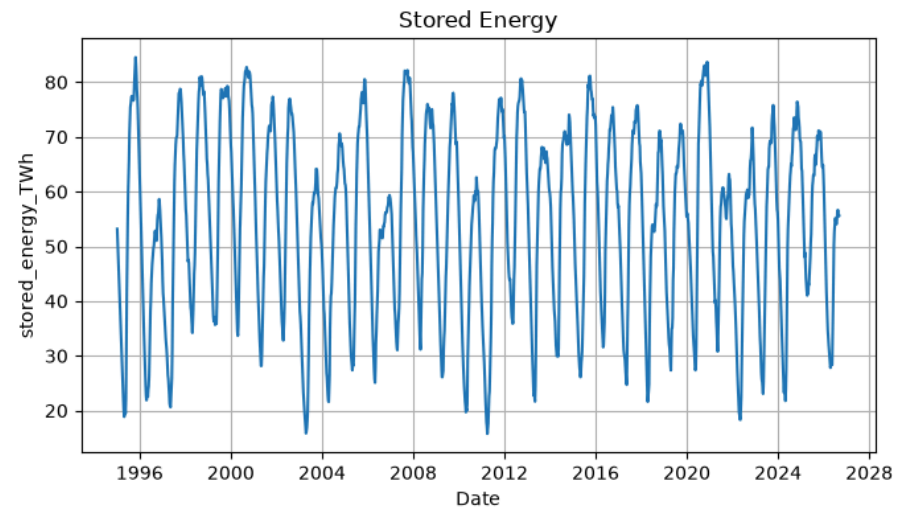
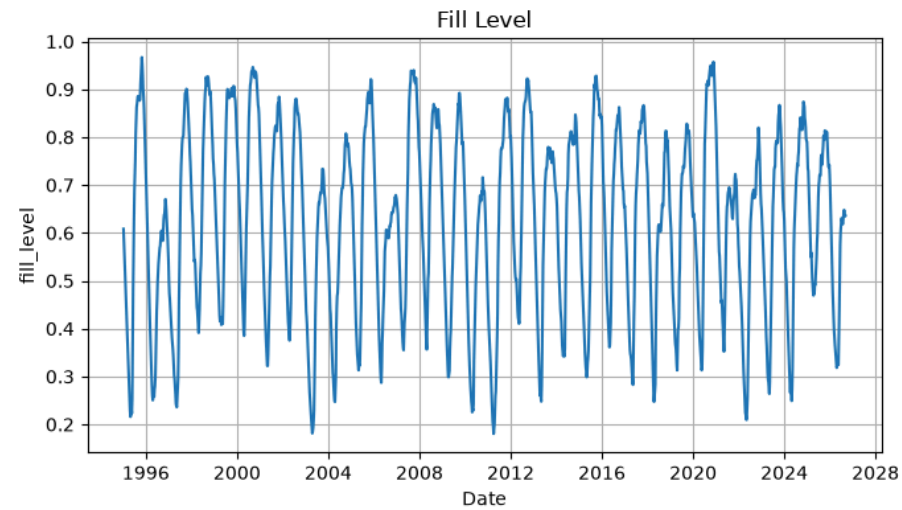
```
In [8]: #Select the national data
df_no = df[(df["area_type"] == "NO")].copy()

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

columns = [
    "fill_level",
    "stored_energy_TWh",
    "fill_level_prev_week",
    "fill_level_change"
]

titles = [
    "Fill Level",
    "Stored Energy",
```

```
    "Fill Level Previous Week",  
    "Weekly Change in Fill Level"  
]  
#Plotting for all the different columns in one plot  
for ax, column, title in zip(axes.flatten(), columns, titles):  
    ax.plot(df_no["date"], df_no[column])  
    ax.set_title(title)  
    ax.set_xlabel("Date")  
    ax.set_ylabel(column)  
    ax.grid()  
  
plt.tight_layout()  
plt.show()
```



Plotting together

```
In [9]: fig, ax1 = plt.subplots(figsize=(14, 6))

#display fill level on the left y-axis
ax1.plot(
    df_no["date"],
    df_no["fill_level"],
```

```
        label="Fill Level",
        color="blue"
    )

    ax1.set_xlabel("Date")
    ax1.set_ylabel("Fill Level", color="blue")
    ax1.grid()

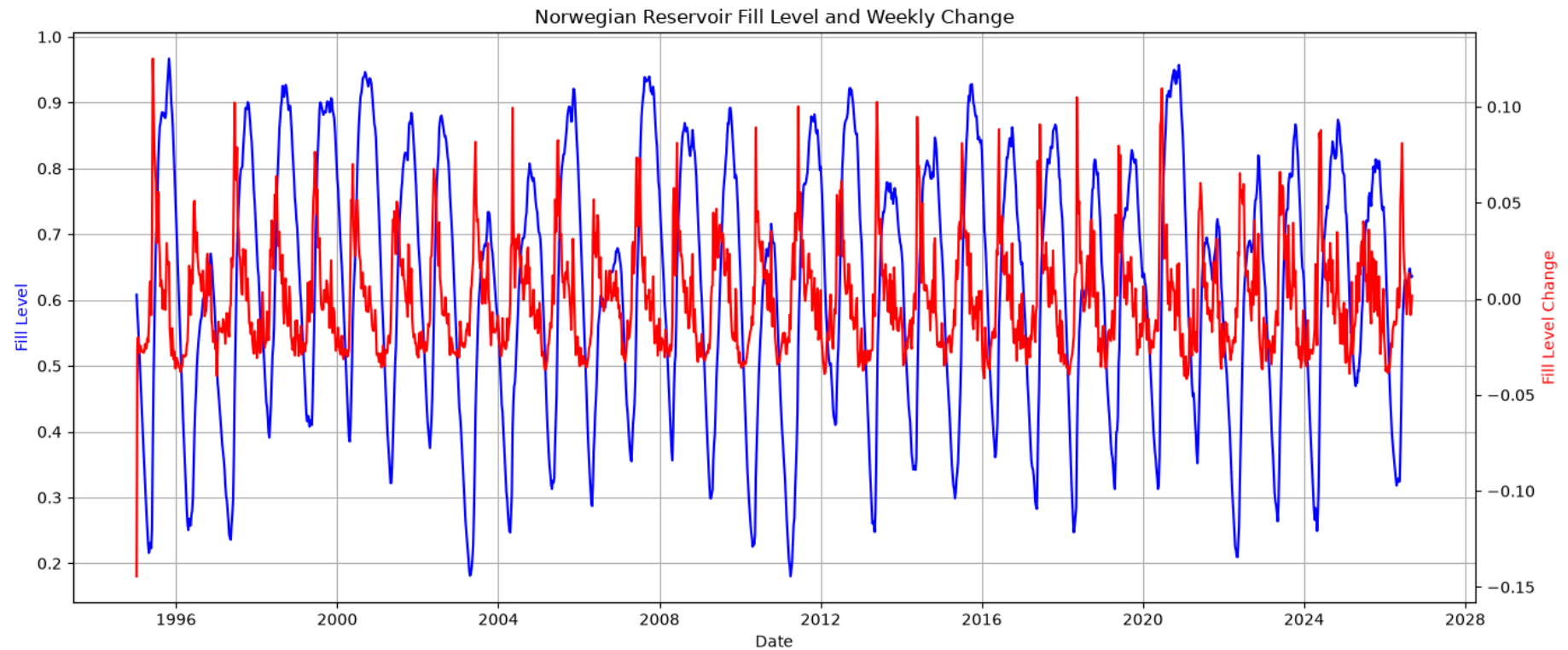
    #display change in fill level on right y-axis
    ax2 = ax1.twinx()

    ax2.plot(
        df_no["date"],
        df_no["fill_level_change"],
        label="Fill Level Change",
        color="red"
    )

    ax2.set_ylabel("Fill Level Change", color="red")

    plt.title("Norwegian Reservoir Fill Level and Weekly Change")

    fig.tight_layout()
    plt.show()
```



AI Usage

I used ChatGPT to help me solve the task. Using AI i could put most of my time and effort into understanding the logic behind the code, and how connections with dictionaries, folder structure etc. worked, and also the streamlit connections. I used AI the most in initializing the project, like basically giving me a step by step guide for installing and setting up everything. For the main task I first asked it to break down / simplify what the specific task parts whas asking for, then explaining it in my own words (logic behind it), before implementing the actual code it gave me. Also the AI was of great help with debugging/troubleshooting, as the python error messages can often be very confusing/unclear.

Log

Started with exploring the dataset. Firstly renamed the columns to feasible english names and also sorted the dataset by date. Found fill_level, stored_energy_TWh, fill_level_prev_week and fill_level_change to be relevant data to show on plots. Decided to use the area_type == "NO" for now, to show the national data, just to make it simpler for display purposes for now. When looking at the fill_level_change i saw that the first recorded values were unusually small number (-0.14) whereas all the others are around a minimum of (-0.05). I checked the calculations myself but it all checked out according to the data so decided to still keep it in. Firstly plotted all of the variables separately. When working with displaying them all together by normalizing, since $\text{stored_energy_TWh} = \text{fill_level} \times \text{capacity_TWh}$, with capacity_TWh being a constant, and that fill_level_prev week is fill_level just shifted one week backwards, i decided that it was reasonable to only plot fill_level and fill_level_change against date. Since fill_level_change contains negative values and fill_level is percentage from 0-1, i used two different y-axis, as normalizing the negative values would destroy its meaning. This provides a much cleaner and visible plot without removing anything important, so i also used this for the "All columns" plot in the streamlit app. Moving on to the streamlit app itself, with emphasis on the "plots" page which contains the most code. Firstly i added a separate "data_loader.py" file under "src" folder. Considering reservoirdata and plots uses the same loading and cleaning of the csv file it would be cleaner to have it as its own program which is imported by the py-files using it. It also contains the caching of the data here. In the plots.py I used the selectbox, select_slider etc with some if tests to make sure the correct selection shows the correct plot. I found it pretty effective and intuitive to work with streamlit in the python code and made it pretty simple to set up the webpage cleanly in my opinion.